PROGRAMMING DSA IN PYTHON part 2 :

Shortest Paths in Weighted Graphs >

1. What are Weighted Graphs?

In standard graphs, an edge simply indicates a connection. In a **weighted graph**, we assign numerical values to these edges to represent things like cost, time, or distance.

To store this data in code, we generally use one of two methods:

- **Adjacency Matrix:** A grid where we record the weights between nodes, usually putting a `0` if no edge exists between them.
- **Adjacency List:** A list where each vertex records its connected neighbors along with the specific weight of that connecting edge.

2. Why Standard BFS Fails Here

Breadth-First Search (BFS) explores a graph level by level, meaning it calculates the shortest path based purely on the *number of edges*.

However, in a weighted graph, the true length of a path is the sum of its edge weights. Therefore, the weighted shortest path does not always have the minimum number of edges.

- **Example from the slides:** To get from node `0` to node `2`, the direct path (1 edge) has a massive weight of 80. But if you route through node `1`, the path takes 2 edges but only costs 16 ($10 + 6$).

3. Types of Shortest Path Problems

Depending on the application, we usually try to solve one of two distinct problems:

- **Single Source:** Finding the shortest path from one specific fixed vertex to every other vertex (e.g., routing trucks from a single distribution center to all retail outlets).
- **All Pairs:** Finding the shortest path between every possible pair of vertices in the graph (e.g., mapping optimal airline or railway routes between all cities).

4. Negative Weights & Negative Cycles

Negative edge weights can actually make sense in the real world. For instance, a taxi driver driving an empty road incurs a cost (positive weight), but driving a road with many customers generates a profit (negative weight).

The problem arises when a graph has a **negative cycle**—a loop where the sum of the edge weights is negative.

- If a negative cycle exists, the shortest path is mathematically undefined. This is because you could just repeatedly drive around that negative loop to infinitely decrease your total cost.
- As long as there are no negative cycles, shortest paths remain well-defined and computable.

Dijkstra's algorithm

1. The Core Problem

Dijkstra's algorithm solves the **Single Source Shortest Path** problem: finding the shortest path from one fixed starting vertex to every other reachable vertex in a weighted graph.

To explain how it works, the professor uses an excellent physical analogy:

- Imagine the vertices are oil depots and the edges are pipelines connecting them.
- The edge weights represent the length of the pipeline.
- You set fire to the starting depot (Node 0) at time $t=0$.
- The fire travels at a uniform speed along every pipeline simultaneously.
- **The fundamental rule:** The first oil depot to catch fire after the start is guaranteed to be the nearest vertex. The next one to catch fire is the second nearest, and so on.

2. How the Algorithm Works (Step-by-Step)

Because we can't actually set our computer on fire, the algorithm simulates this process using two main lists (or dictionaries):

- `visited`: A boolean tracker of whether a node is fully "burnt" (initially `False` for all).
- `distance`: The "expected burn time" or shortest known distance to a node (initially set to infinity `∞` for all, except the start node which is `0`).

The Loop:

1. Look at all unvisited nodes and pick the one with the smallest `distance`.
2. Mark this node as `visited` (it has caught fire!).
3. Look at all the unvisited neighbors of this newly burnt node. Calculate if traveling through this new node offers a shorter path than their current `distance`. If it does, update their `distance`.

4. Repeat until all reachable vertices are visited.

3. The Crucial Limitation: No Negative Weights

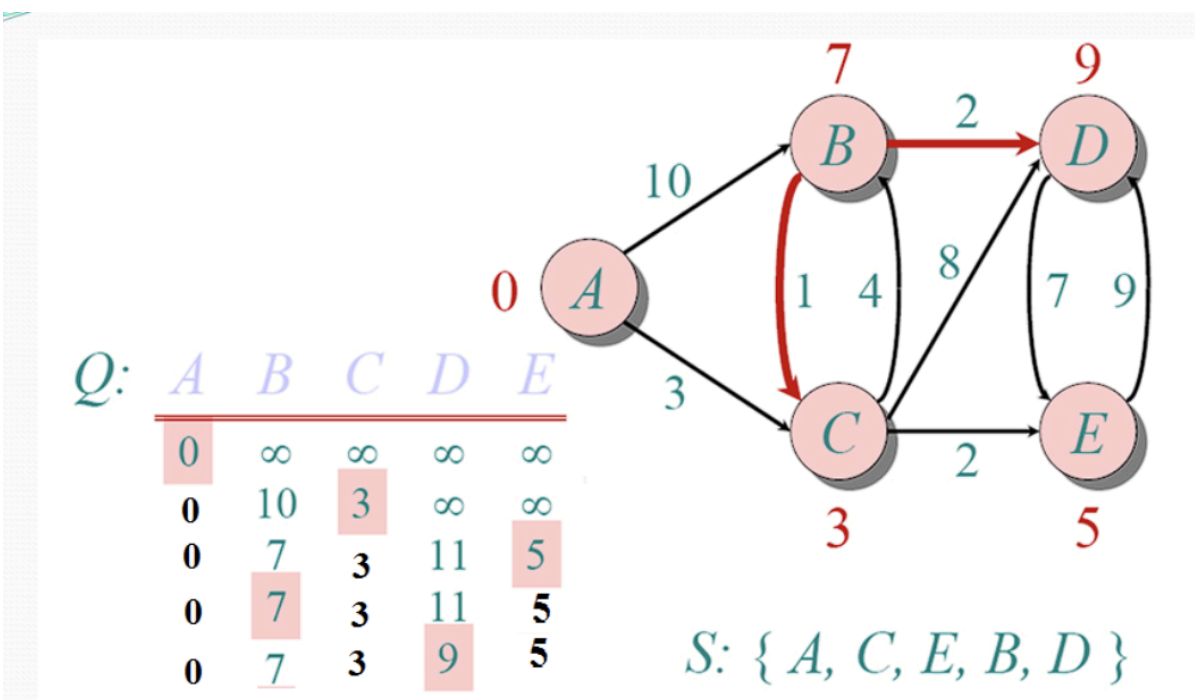
Dijkstra's algorithm has one massive vulnerability: **it completely fails if there are negative edge weights**.

Why? Think back to the fire analogy. When a depot catches fire, the algorithm locks in its shortest path and assumes it will never find a shorter route to that depot later. A negative weight would be like a pipeline that causes the fire to travel *backwards in time*, arriving earlier than it started. The algorithm's greedy assumption breaks down entirely.

4. Time Complexity

The algorithm's speed depends on how it is implemented:

- **Adjacency Matrix:** The overall complexity is $O(n^2)$. The main loop runs n times. Inside that loop, it takes $O(n)$ time to scan the array to find the next minimum vertex, and $O(n)$ time to update the neighbors.
- **Adjacency List:** You might think this is faster, but standard lists still result in an $O(n^2)$ bottleneck. Even though updating neighbors is faster, you still have to scan through a list of size n every single time to find the minimum unvisited vertex.
- *Future Note:* The slides hint that a "better data structure" is needed. (Spoiler: Using a Priority Queue/Min-Heap will fix this bottleneck in upcoming lectures!).



Bellman ford algo

1. Why Dijkstra's Algorithm Fails with Negative Weights

Dijkstra’s Algorithm relies on a "greedy" strategy, assuming that edge weights are always non-negative.

- **The Problem:** In Dijkstra's, once a vertex "burns" (is visited), its shortest distance is permanently frozen. The algorithm assumes it will never find a shorter route to that vertex later.
- **The Breakdown:** If an edge has a negative weight, it acts like a "shortcut" that decreases the total path cost. Dijkstra's algorithm might miss these shortcuts because it stops updating a node's distance once it is marked as burnt.

2. The Bellman-Ford Approach (Edge Relaxation)

The Bellman-Ford algorithm solves this by allowing distance updates *even after* a node has been reached.

- **The Core Logic:** Every prefix of a shortest path must itself be a shortest path. The algorithm iteratively updates the shortest distance to each vertex based on the shortest distances of its neighbors.
- **The Maximum Path:** In any graph without negative cycles, a shortest path to any node will contain at most $n-1$ edges (where n is the total number of vertices).
- **The Loop:** Therefore, instead of picking nodes one by one like Dijkstra, Bellman-Ford simply takes *every single edge* in the entire graph and tries to "relax" it (find a shorter path). It repeats this full graph sweep exactly $n-1$ times. After $n-1$ updates, all shortest paths are mathematically guaranteed to have stabilized.

3. Handling Negative Cycles

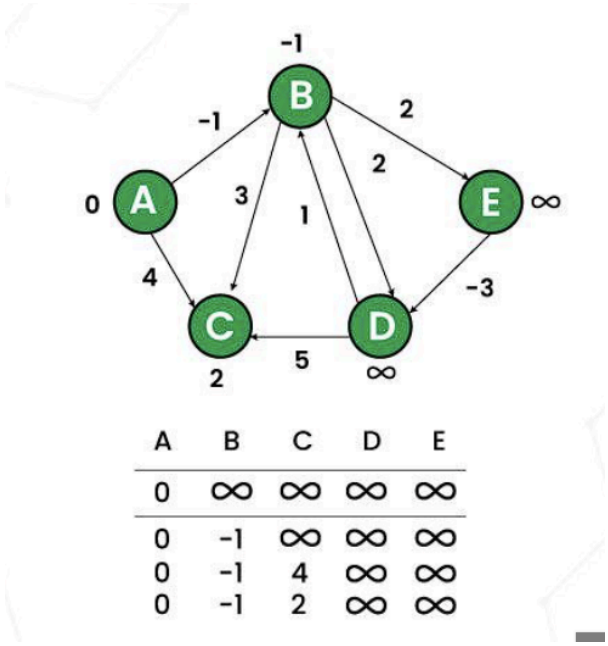
While negative *weights* are fine, **negative cycles** (a loop where the total weight is less than zero) break the concept of a shortest path.

- If a negative cycle exists, you could infinitely loop through it, meaning the distance would continuously decrease forever.
- **Detection:** Bellman-Ford detects this easily. If you run a final n-th iteration and any node's distance *still* decreases, it means the graph contains a negative cycle.

4. Time Complexity

Because Bellman-Ford checks every edge repeatedly, it is slower than Dijkstra's Algorithm.

- **Using an Adjacency Matrix:** Examining every edge takes $O(n^2)$ time per iteration. Since the outer loop runs $O(n)$ times, the overall complexity is $O(n^3)$.
- **Using an Adjacency List:** Scanning all edges takes $O(m)$ time per iteration (where m is the number of edges). The overall complexity improves slightly to $O(mn)$.



Floyd-Warshall Algorithm for finding All-Pairs Shortest Paths.

---

1. The "All-Pairs" Problem

So far, we have looked at **Single-Source** algorithms (Dijkstra's and Bellman-Ford), which calculate the shortest path from *one* specific starting node to all other nodes.

But what if you need a complete distance chart, like a mileage table between every major city on a map? You need the shortest path between **every possible pair** of vertices.

- *Naive Solution:* You could just run Bellman-Ford or Dijkstra's algorithm V times (once for every single vertex as the starting point). However, this is computationally expensive.
- *The Better Way:* The **Floyd-Warshall Algorithm** solves this entirely differently, evaluating the graph holistically using Dynamic Programming.

2. The Core Logic: Intermediate Nodes

Floyd-Warshall operates directly on an **Adjacency Matrix** (a 2D grid representing the graph).

Instead of moving "level by level" or "edge by edge", it asks one simple question repeatedly: **Can I find a shorter path from Node A to Node B by taking a detour through Node K?**

How it works step-by-step:

1. **Initialize:** Start with your base Adjacency Matrix. The distance from a node to itself is `0`. The distance to direct neighbors is the edge weight. The distance to non-neighbors is `Infinity`.
2. **Iterate through Intermediaries (k):** The algorithm uses a loop to temporarily pick every node in the graph, one by one, to act as an "intermediary" or "detour" node (k).
3. **Check Every Pair (i to j):** While focusing on detour node k, the algorithm checks every possible pair of nodes (i and j).
4. **The Formula:** It compares the current known distance of `i -> j` against the detour distance `(i -> k) + (k -> j)`. If the detour is cheaper, it updates the matrix with the new, lower cost.

3. Negative Weights & Cycles

Because Floyd-Warshall doesn't "lock in" distances permanently like Dijkstra's, it **can perfectly handle negative edge weights**.

Detecting Negative Cycles:

It is also incredibly easy to detect negative cycles using this algorithm. Initially, the distance from any node to itself (the main diagonal of the matrix) is set to `0`. If, at the end of the algorithm, any node has a distance to itself that is **less than zero**, it absolutely guarantees that a negative cycle exists in the graph.

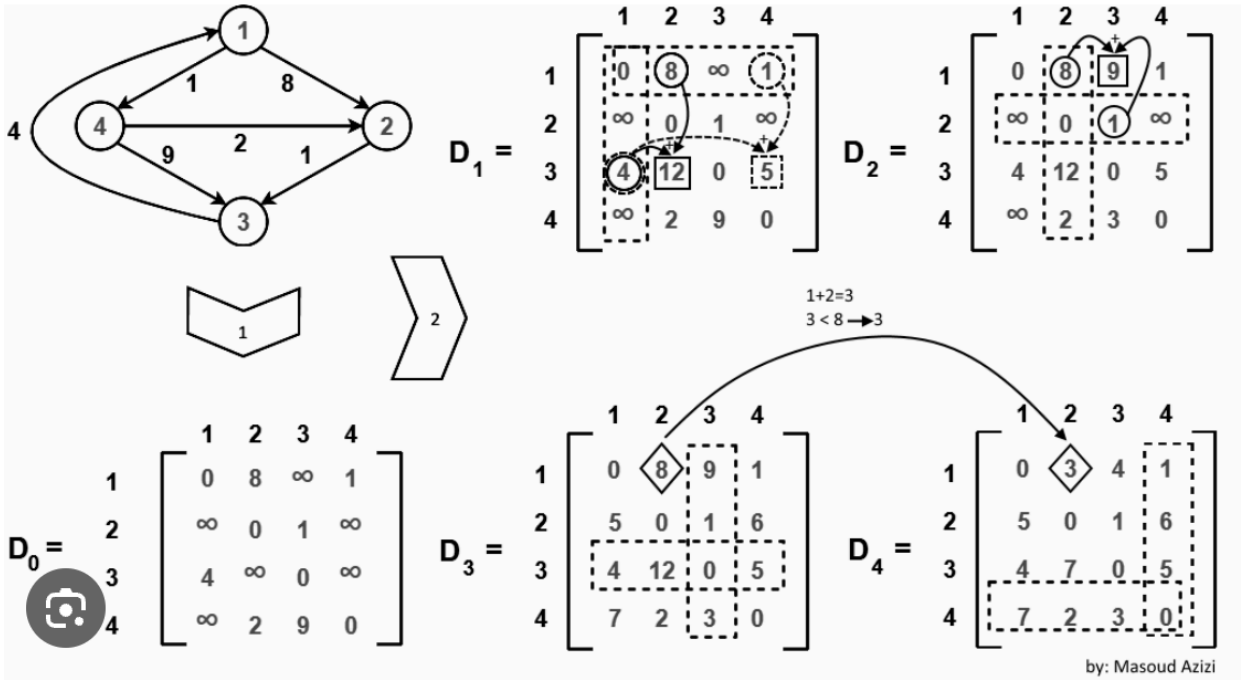
4. Time Complexity

The algorithm is famous for its elegant, extremely short code—it is literally just three nested loops:

Python:

```
# The heart of Floyd-Warshall
for k in range(V):          # The detour node
        for i in range(V):      # The start node
                for j in range(V):  # The end node
                        distance[i][j] = min(distance[i][j], distance[i][k] + distance[k][j])
```

- **Complexity:** Because of the three nested loops running V times, the time complexity is strictly **$O(V^3)$**, regardless of how many edges the graph has.
- **When to use it:** It is fantastic for dense graphs (graphs with many edges) represented as adjacency matrices.



Minimum Cost Spanning Trees (MCST),

1. What is a Spanning Tree?

Before finding the *minimum* cost, we have to understand what a spanning tree is.

Imagine a connected, undirected graph with V vertices. A **spanning tree** is a subgraph that satisfies three rules:

1. It must include **every single vertex** from the original graph.
2. It must be fully connected (you can reach any node from any other node).
3. It must contain **absolutely no cycles** (no loops).

Because of these rules, every spanning tree will always have exactly $V-1$ edges.

2. The Minimum Cost Spanning Tree (MCST)

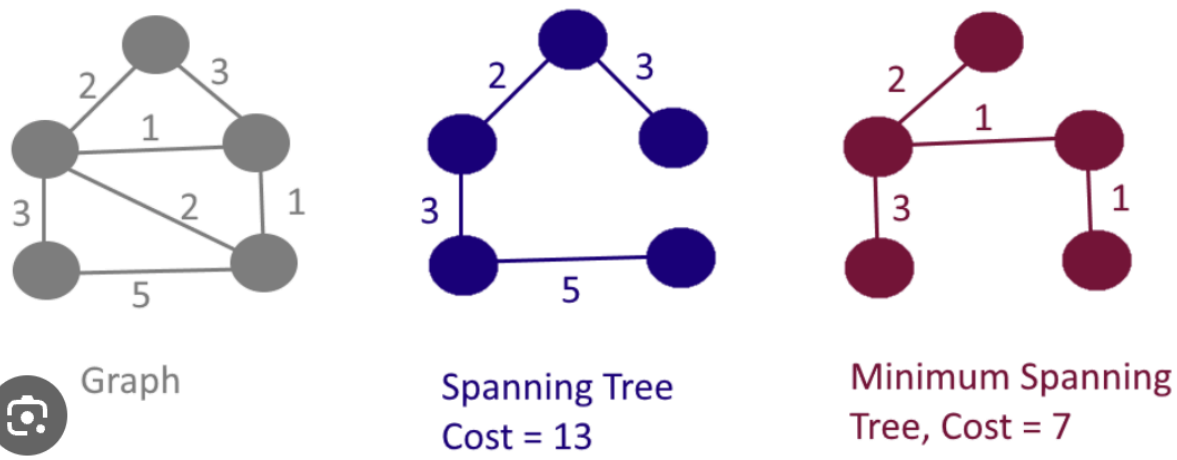
In a weighted graph, different spanning trees will have different total costs (the sum of all their edge weights). The **Minimum Cost Spanning Tree** is the spanning tree where this total sum is as small as possible.

- **Real-World Application:** Imagine the vertices are cities and the edges are potential routes for laying fiber-optic internet cables. The weights represent the cost of laying cable on that route. An MCST gives you the exact layout needed to connect every city to the network for the absolute lowest total cost.

3. How to Find the MCST

There are two famous "greedy" algorithms used to solve this problem efficiently:

- **Prim's Algorithm:** * **How it works:** It starts at an arbitrary "root" vertex and grows the tree one step at a time. At every step, it looks at all the edges connecting the current tree to the unvisited nodes and greedily picks the edge with the lowest weight.
    - **Similarity:** It feels very similar to Dijkstra's algorithm because it builds outward from a single source.
- **Kruskal's Algorithm:**
    - **How it works:** It ignores the concept of a "starting node." Instead, it sorts every edge in the entire graph from lowest weight to highest weight. It then adds edges to the network one by one, strictly skipping any edge that would accidentally create a cycle, until all vertices are connected.



Prim's Algorithm for finding Minimum Cost Spanning Trees.

---

1. The Core Strategy: Growing the Tree

Prim’s algorithm is a greedy algorithm that builds the Minimum Spanning Tree (MST) one vertex at a time. It feels very similar to a mold or a network slowly expanding outward.

- **The Rule:** You start with a single, arbitrary "root" vertex. Then, you look at all the edges that connect your current tree to the unvisited nodes (the "fringe" or "boundary"). You strictly pick the edge with the **minimum weight**, add that node to your tree, and repeat.

2. The Tracking Mechanism

Because Prim's algorithm explores outward from a starting point, it uses a tracking system almost identical to Dijkstra's algorithm. It maintains:

- `visited`: A list to track which nodes are already part of the spanning tree.
- `distance`: This tracks the **cheapest known edge** connecting an unvisited node directly to *any* node currently in the tree (initialized to ∞).
- `parent` (or `TreeEdges`): A list that remembers *which* specific edge was used to connect the node, so you can actually draw the final tree at the end.

3. Prim's vs. Dijkstra's (The Crucial Difference)

These two algorithms look nearly identical in code, but they answer different questions, which changes one vital line of math:

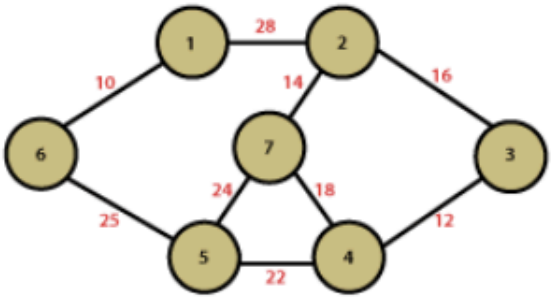
- **Dijkstra's (Shortest Path):** You care about the total distance from the *start node*.
  - *Update Rule:* `distance[v] = min(distance[v], distance[u] + weight(u, v))`
  - *(Current total path cost + new edge)*
- **Prim's (Minimum Spanning Tree):** You don't care about the start node anymore; you only care about attaching a new node to the *current tree* as cheaply as possible.
  - *Update Rule:* `distance[v] = min(distance[v], weight(u, v))`
  - *(Just the raw cost of the new edge)*

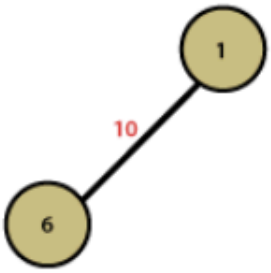
4. Time Complexity

Because the structure is the same as Dijkstra's, the time complexity is also the same:

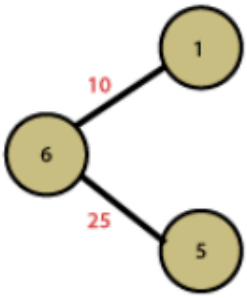
- **Using an Adjacency Matrix:** $O(V^2)$. In every step, you have to scan the unvisited nodes to find the minimum distance, which creates a bottleneck.
- *Note:* Just like Dijkstra, this can be heavily optimized using a Priority Queue (Min-Heap) if you use an Adjacency List.

Prim's Algorithm

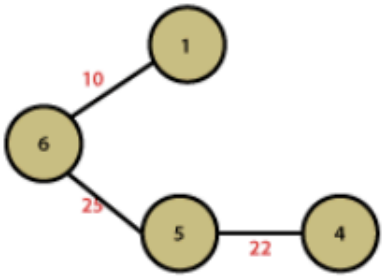




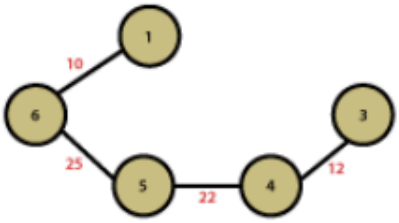
Step 1



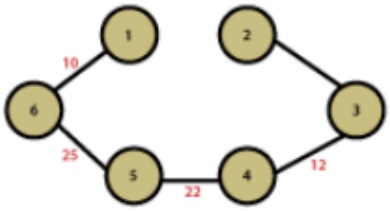
Step 2



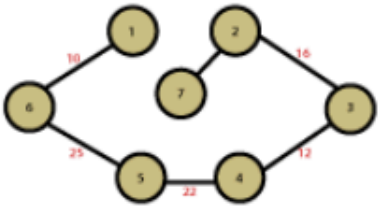
Step 3



Step 4



Step 5



Step 6

Kruskal's Algorithm for finding Minimum Cost Spanning Trees.

*(To answer the cliffhanger question from the last lecture: **Yes!** If a graph has multiple edges with the exact same weight, the Minimum Cost Spanning Tree is not uniquely determined. Imagine a simple triangle with three vertices where every edge weighs `1`. Any two edges will form a valid spanning tree with a total cost of `2`—meaning there are three different, perfectly valid MCSTs!)*

---

1. The Core Strategy: The Bottom-Up Approach

While Prim's algorithm starts at one node and grows a single tree outward like a mold, Kruskal's algorithm builds the MCST from the "bottom up".

- **The Setup:** At the very beginning, Kruskal's completely ignores the structure of the graph. It takes every single edge, throws them into a list, and **sorts them in ascending order of cost**.
- **The Islands:** It treats all n vertices as completely isolated "components" (or islands).
- **The Rule:** You go down your sorted list of edges one by one. Whenever an edge connects two completely separate, disjoint components, you permanently add it to the MCST and merge those two components into one. If an edge tries to connect two vertices that are *already* in the same component, you throw it away, because that would form a cycle.

2. The Implementation Challenge (Cycle Detection)

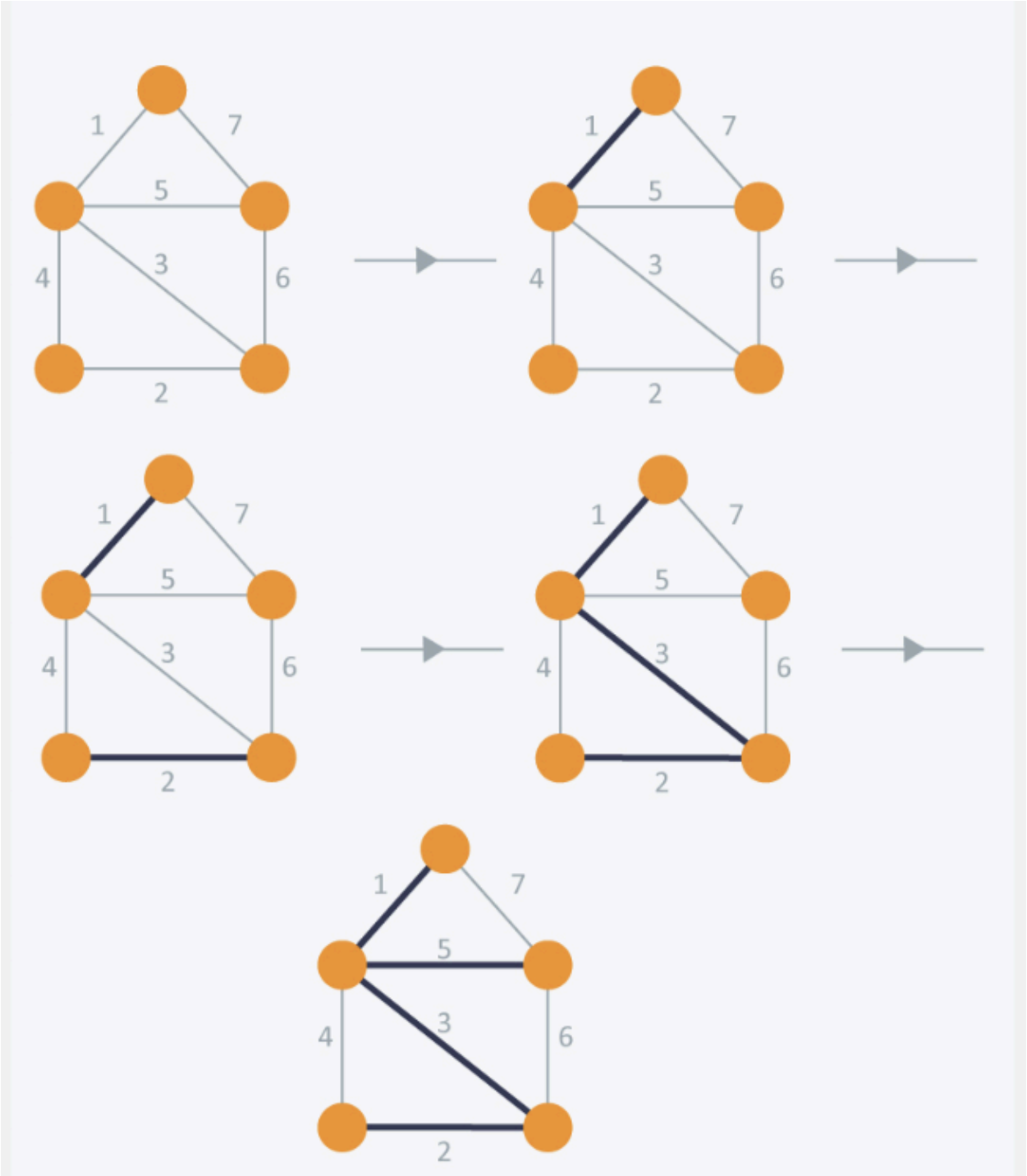
The hardest part of Kruskal's algorithm is figuring out if an edge creates a cycle. How does a computer actually know if two nodes are already in the same component?

- **The Naive Approach:** You can create an array called `component` where each vertex is assigned a unique component ID (e.g., `component[0] = 0`, `component[1] = 1`).
- **Checking for Cycles:** When evaluating an edge between node u and node v, you just check if `component[u] == component[v]`. If they match, it's a cycle. Skip it.
- **Merging:** If they don't match, you add the edge to the tree. Then, you have to merge them by updating the component array. If `component[u]` is `c` and `component[v]` is `c'`, you must scan the *entire array* and change every single `c` to a `c'`.

3. Time Complexity

Because of that naive merging step, the time complexity has a severe bottleneck:

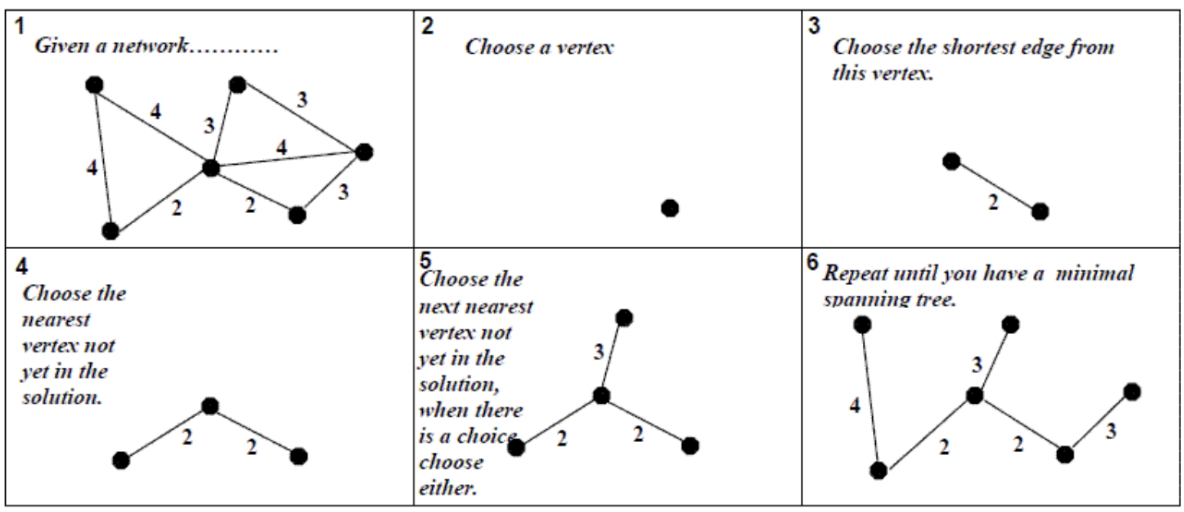
- **Sorting the Edges:** Takes $O(m \log m)$ time (where m is the number of edges).
- **Merging Components:** Scanning the array of size n every time you add an edge takes $O(n)$ time. Since a spanning tree has $n-1$ edges, this part takes $O(n^2)$ time overall.
- *Future Optimization Note:* The lecture mentions that this $O(n^2)$ bottleneck can be drastically improved to $O(m \log n)$. (This is traditionally done using a clever data structure called "Union-Find" or "Disjoint Set", which you will likely cover soon).


1
7
5
4
3
6
2
1
7
5
4
3
6
2
1
7
5
4
3
6
2
1
7
5
4
3
6
2
1
7
5
4
3
6
2

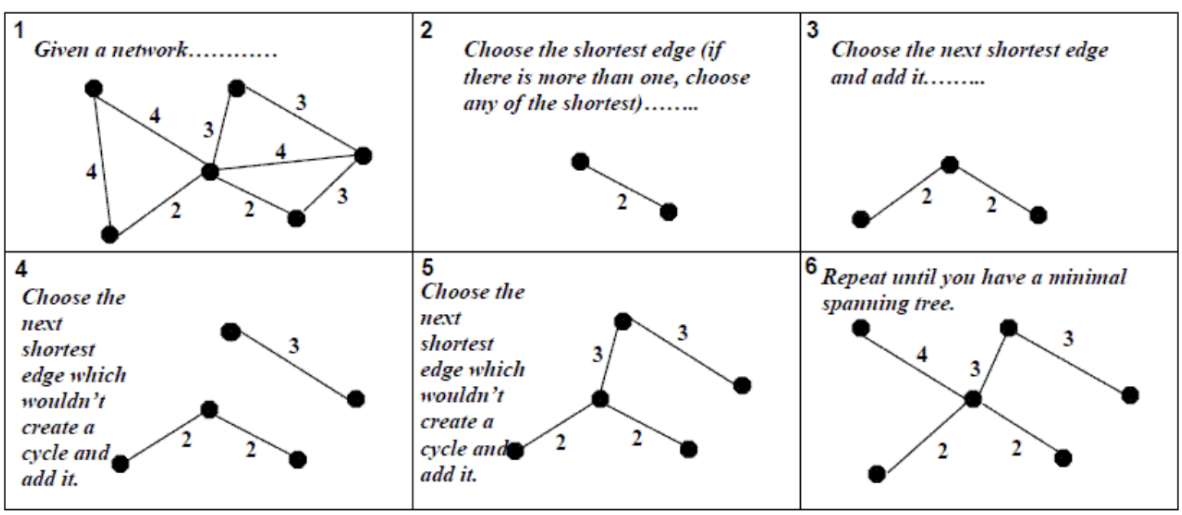
Prim's Algorithm	Kruskal's Algorithm
Dense Graphs	Sparse Graphs
Starts with a node	Sorted list of edges
Priority Queue and Visited Array	Union Find
Time Complexity: O (E log V)	Time Complexity: O (E log E)
Expands like Dijkstra's Algorithm	Greedy on Sorted edges
Adjacency List and Priority Queue	Edge List and Union Find

Algorithm	Time Complexity
Kruskal's Algorithm	O(E Log E or E Log V)
Prim's Algorithm	O(E log V) using binary Heap / O(E + V log V) using Fibonacci Heap

Prim's Algorithm



Kruskal's Algorithm



Union-Find Data Structure, which is the crucial missing piece for optimizing Kruskal's algorithm.

---

1. The Problem with Naive Components

In Kruskal's algorithm, we process edges in ascending order of cost and add them to the spanning tree as long as they don't create a cycle. To prevent cycles, we only add an edge (u, v) if u and v are currently in completely different components.

If they are in different components, adding the edge merges them together. To do this efficiently, we partition the vertices into a collection of "disjoint sets". We need a data structure that supports three core operations:

- **MakeUnionFind(S):** Initializes the structure where each vertex is in its own isolated component.
- **Find(v):** Returns the specific set/component that contains vertex v.
- **Union(u, v):** Merges the two sets containing u and v into a single component.

2. The Naive Implementation (O(n) Bottleneck)

The simplest way to implement this is using an array (or dictionary) called `Component`, where `Component[i]` stores the ID of the set containing vertex `i`.

- **Find:** Incredibly fast. Just look up `Component[i]`, which takes $O(1)$ time.
- **Union:** Very slow. If you want to merge Component $C1$ and Component $C2$, you have to scan the *entire* array to find every single element belonging to $C1$ and change its value to $C2$. This takes $O(n)$ time.

3. The Smart Optimization: Merge by Size

To fix the $O(n)$ bottleneck, we can maintain two extra pieces of information for each component: a list of its `Members` and its total `Size`.

The Rule: When performing a `Union`, **always merge the smaller component into the larger component**.

Instead of scanning the whole array, you look at the `Members` list of the smaller component, and strictly update the `Component` ID for only those few elements. You then append those members to the larger component's list and update the size.

4. Amortized Complexity Analysis

Why does "merging by size" make such a huge difference?

- An element's component ID is only updated when its component is the *smaller* of the two being merged.
- Every time an element's component ID gets updated, the size of its new merged component is at least **double** the size of its old component.

- Because the size doubles every time, an element's ID can only be updated at most $\log_2 n$ times before it is part of a component containing all n elements.

Therefore, across a sequence of m Union operations, an individual element is updated at most $O(\log n)$ times. This makes the **amortized (average) cost of each Union operation $O(\log n)$**.

5. Kruskal's Final Speed

With this optimized Union-Find structure, Kruskal's algorithm becomes incredibly fast:

1. Sorting the edges takes $O(m \log m)$.
2. The $n-1$ Union operations take an amortized total of $O(n \log n)$.
3. Because the maximum number of edges $m \le n^2$, $O(m \log m)$ is equivalent to $O(m \log n)$.

Overall time for Kruskal's Algorithm is improved to **$O(m \log n)$**.

Priority Queue : to finding an efficient way to implement it.

---

1. The Problem: Job Scheduling

Traditional queues operate on a "First-In, First-Out" (FIFO) basis. However, in the real world, tasks rarely have equal importance.

Imagine a computer's Job Scheduler:

- It maintains a list of pending jobs, each with a specific priority level.
- Whenever the processor is free, the scheduler must pick the job with the **maximum priority** from the list and run it.
- To make things harder, new jobs can join the list at any random time.

To handle this, we need a data structure that supports two core operations efficiently: `insert()` and `delete_max()`.

2. The Naive Approaches (Lists/Arrays)

If we try to build a Priority Queue using basic lists, we run into a bottleneck.

- **Unsorted List:** * `insert()` is extremely fast ($O(1)$) because you just append the new job to the end.
    - `delete_max()` is very slow ($O(N)$) because you have to scan the entire list to find the largest number.
- **Sorted List:**
    - `delete_max()` is extremely fast ($O(1)$) because the largest number is always sitting at the very end of the list.
    - `insert()` is very slow ($O(N)$) because you have to scan the list to find the correct sorted spot, and then shift all the other elements over to make room.

If you have N jobs to process, both of these simple structures will take **$O(N^2)$** total time. We need something better.

3. The Intermediate Step (2D Grid)

The lecture introduces a clever, intermediate mathematical trick to speed things up: the $2D \sqrt{N} \times \sqrt{N}$ array.

Instead of one long list, imagine breaking N jobs into a square grid.

- You keep the rows sorted.

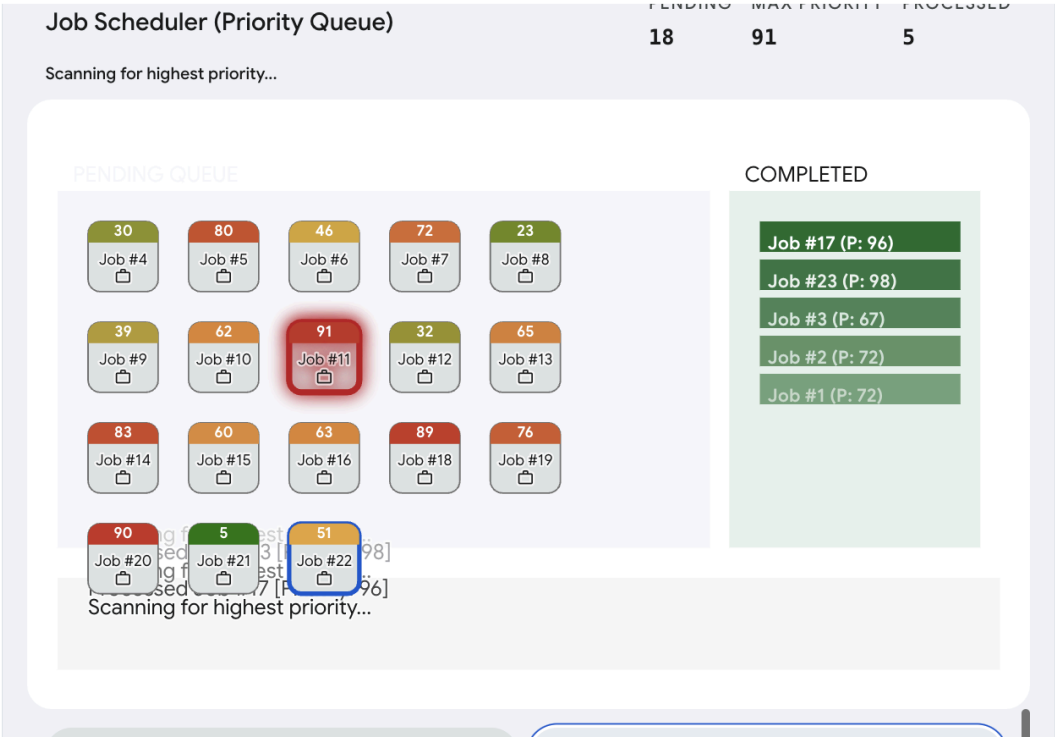
- **`delete_max()`:** Since the rows are sorted, the maximum element of each row is at the end. You just check the ends of the $\sqrt{N}$ rows to find the absolute maximum. This takes $O(\sqrt{N})$ time.
- **`insert()`:** You find the row with the fewest elements and insert the job in its sorted position. This also takes $O(\sqrt{N})$ time.

Processing N items with this grid drops the total time from $O(N^2)$ to **$O(N\sqrt{N})$**. But we can still do better.

4. The Ultimate Solution: Heaps

To get the absolute best performance, computer scientists use a special data structure called a **Heap**.

- A Heap is a specialized **binary tree**.
- Because it is a balanced tree, its height is strictly limited to $O(\log N)$.
- By organizing the tree so that the largest elements naturally float to the top, both `insert()` and `delete_max()` can be performed by tracing a path down the tree, which takes **$O(\log N)$** time.
- Processing N items with a Heap takes only **$O(N \log N)$** time.



Heaps, the ultimate data structure for implementing Priority Queues.

---

1. The Problem with Lists and Grids

As discussed in the previous lecture, maintaining a priority queue using basic lists incurs a massive cost. Across N insertions and deletions, a standard list takes $O(N^2)$ time.

While using a 2D $\sqrt{N} \times \sqrt{N}$ array reduces the cost to $O(\sqrt{N})$ per operation (resulting in $O(N\sqrt{N})$ overall), we can achieve even better performance.

2. The Ultimate Solution: The Heap

A **Heap** is a specialized binary tree implementation of a priority queue.

- **The Shape:** It is a *complete* binary tree, meaning it is perfectly balanced and filled from left to right. Because of this strict shape, its height is always kept to $O(\log N)$.
- **The Rule (Max-Heap):** Every parent node must have a priority greater than or equal to both of its children. This mathematically guarantees that the absolute maximum item is always sitting right at the root of the tree.
- **The Speed:** Because the tree height is $O(\log N)$, tracing a path from the root to the bottom is incredibly fast. Both the `insert()` and `delete_max()` operations take only **$O(\log N)$** time.

3. Core Operations: Bubble Up & Bubble Down

While the slides don't detail the exact mechanics of these operations, here is how the heap maintains its strict rule in $O(\log N)$ time:

- **`insert()`:** You add the new item to the very bottom of the tree. If it is larger than its parent, they swap places (it "bubbles up" the tree) until it reaches a valid spot.
- **`delete_max()`:** You remove the root (the highest priority item). To fill the hole, you take the very last item at the bottom of the tree and move it to the root. If this item is smaller than its new children, it swaps with the largest child (it "bubbles down") until it settles.

4. The Magic of `heapify()`: Building a Heap in $O(N)$ Time

If you have a random, unsorted list of N items, how do you turn it into a valid Heap?

- *Naive Way:* You could just run `insert()` N times. Since each insert takes $O(\log N)$ time, this would take $O(N \log N)$ total time.
- *The Smart Way (`heapify()`):* You can actually build a heap in **$O(N)$** time.

How `heapify()` works:

1. Start with the array $L=[v_0, v_1, ..., v_N]$.
2. Find the midpoint: `mid = len(L)//2`.

3. The second half of the array (`Slice L[mid:]`) represents the "leaf" nodes at the very bottom of the tree. A single node with no children is already a valid heap!
4. Working backward from the middle of the array up to the root, you "fix the heap property downwards" level by level.
5. **The Math:** Each time we go up one level, the maximum number of steps required to bubble down increases by 1, but the number of nodes we have to check *halves*. (e.g., The second last level has $n/4$ nodes requiring 1 step, the third last has $n/8$ nodes requiring 2 steps, etc.). This shrinking geometric series mathematically resolves to exactly $O(N)$ cost.

---

covers how to practically apply Heaps to supercharge algorithms like Dijkstra's and Prim's.

---

1. The Bottleneck: Finding the Minimum

In our previous look at Dijkstra's and Prim's algorithms, we saw a glaring performance issue. In a graph with n vertices, the algorithm loops n times. Inside that loop, it scans all unvisited vertices to find the one with the absolute minimum distance, which takes $O(n)$ time.

This $O(n) \times O(n)$ nested loop creates an **$O(n^2)$ bottleneck**. We know Heaps (Priority Queues) can find and delete the minimum element in $O(\log n)$ time. So, why don't we just use a Heap?

2. The Missing Operation: Updating Values

Standard Priority Queues only support two things: `insert()` and `delete_min()`.

However, in Dijkstra's algorithm, when you evaluate a neighbor, you often find a shorter path to it. This requires you to **update (decrease) the distance of a vertex that is already sitting inside the heap**.

- **The Problem:** A heap is not a search tree. Finding a specific vertex inside a standard heap takes $O(n)$ time because you have to check every single node. If finding the node takes $O(n)$, using a heap doesn't actually save us any time!

3. The Fix: Two-Way Mapping

To solve this, we must extend the standard heap by maintaining additional pointers.

We create a mapping (like a dictionary or array) that tracks the exact position of every vertex in the heap.

- `Heap_Array[5]` might contain `Vertex B`.
- Therefore, `Position_Map[Vertex B] = 5`.

Now, if we need to update Vertex B, we don't have to search the heap. We look at our map, instantly go to index 5 ($O(1)$ time), and update its value.

> *Crucial Detail:* Whenever two nodes swap places in the heap during a bubble-up/bubble-down, we **must** also update their indexes in the `Position_Map`.

4. The Update Process (Decrease-Key)

Once we instantly find the node using our map and decrease its distance, it might now be smaller than its parent. To restore the Min-Heap property, we simply let it **"bubble up"** the tree until it settles.

- Updating a value using this method takes **$O(\log n)$ time**.

5. The Final Speed Boost

With this extended heap, Dijkstra's (and Prim's) algorithms become incredibly fast:

- We do n `delete_min()` operations: $O(n \log n)$.
- We do at most m (number of edges) updates: $O(m \log n)$.
- **Total Complexity:** The algorithm improves from $O(n^2)$ down to **$O((m + n) \log n)$**. For sparse graphs where there are far fewer edges than n^2, this is a massive upgrade!

Binary Search Trees (BSTs), which are used to maintain dynamic sorted data.

---

1. The Motivation: Dynamic Sorted Data

If you have a sorted list (array) of numbers, searching it is very fast (using binary search takes $O(\log n)$ time). However, if your data is *dynamic*—meaning you are constantly inserting new items and deleting old ones—a sorted list is terrible. Inserting or deleting an element in a sorted list takes $O(n)$ time because you have to shift all the other elements over to make room or fill the gap.

To fix this, we move to a tree structure.

2. The Binary Search Tree (BST) Rule

A Binary Search Tree is a binary tree that strictly enforces one simple rule for every single node in the tree:

- All values in the **left subtree** must be strictly **less than** the node's value.
- All values in the **right subtree** must be strictly **greater than** the node's value.
  (Note: For simplicity, this implementation assumes there are no duplicate values).

Because of this rule, finding the absolute minimum or maximum value in the entire tree is trivial:

- **Minimum:** Keep travelling down the left-most path until you hit a dead end.
- **Maximum:** Keep travelling down the right-most path until you hit a dead end.

3. Core Operations

- **Find `find(v)`:** Start at the root. If `v` matches the root, you are done. If `v` is smaller, search the left child. If `v` is larger, search the right child. If you hit an empty spot (None), the value is not in the tree.
- **Insert `insert(v)`:** Use the exact same logic as `find()`. Traverse down the tree comparing `v` to the nodes. If you eventually hit a dead end (an empty spot), that is exactly where you insert the new node `v`.
- **Delete `delete(v)`:**
  Deletion is the most complex operation because you cannot break the BST rule. There are three cases:
  1. **Leaf Node (No children):** Simple. Just delete it and make its spot empty.
  2. **One Child:** Also relatively simple. Delete the node and "promote" its single child (and its subtrees) up to take its exact place.
  3. **Two Children:** You can't just delete it because you'd have two orphaned subtrees. Instead, you find the **Maximum value in the left subtree**. You copy that maximum value into the node you want to delete. Then, you simply delete that original maximum node from the left subtree.

4. Time Complexity & The Balance Problem

The speed of `find`, `insert`, and `delete` entirely depends on the height (h) of the tree. Every step down the tree takes $O(1)$ time, so the total time is proportional to the height ($O(h)$).

- **Best Case (Balanced):** If the tree is perfectly balanced (looks like a full triangle), the height is $O(\log n)$. This makes all operations extremely fast!
- **Worst Case (Unbalanced/Skewed):** What happens if you insert already sorted data (e.g., 1, 2, 3, 4, 5)? Every new node will just be attached to the right child of the previous one. The tree becomes a single straight line. The height becomes $O(n)$, and your operations degrade to $O(n)$—no better than a standard list!

Balanced Search Trees, which addresses the massive performance vulnerability found in standard Binary Search Trees (BSTs).

---

1. The Problem: The $O(n)$ Nightmare

In the previous lecture, we established that operations on a Binary Search Tree (`find()`, `insert()`, and `delete()`) all involve walking down a single path from the root to a leaf.

Therefore, the worst-case time for any of these operations is directly proportional to the **height of the tree**.

- **The Ideal Case:** A perfectly balanced tree has a height of $O(\log n)$, making operations lightning-fast.
- **The Worst Case:** If you insert data that is already sorted (e.g., 1, 2, 3, 4, 5), the tree degrades into a straight line. The height becomes $O(n)$.

To fix this, we need a mechanism to **maintain balance** automatically as the tree grows and shrinks.

2. Defining "Balance"

How does a computer mathematically know if a tree is balanced? The general rule is that the left subtree and the right subtree of any given node should be roughly "equal".

There are two primary ways to measure this equality:

1. **Size:** Comparing the total number of nodes in the left subtree vs. the right subtree.
2. **Height (The Standard Approach):** Comparing the maximum depth of the left subtree vs. the right subtree. The difference between these two heights is often called the **"slope"** or "balance factor".

The Golden Rule of AVL Trees (a specific type of balanced tree): The height of the left subtree and the right subtree can never differ by more than 1.

3. The Implementation Bottleneck: Computing Height

To check if a node is balanced, we need to know the height of its subtrees.

- **The Naive Way:** You could write a `height()` function that recursively explores the entire tree to find the deepest node. However, this traversal takes $O(n)$ time. If you run an $O(n)$ height check every time you insert a node, your insertion time becomes terribly slow!
- **The Smart Way (Caching):** Instead of calculating the height from scratch every time, we add a new variable `self.height` directly to the `Tree` class.

4. Updating Height on the Fly

When you insert a new node, you travel down the tree, and then the recursive functions return back *up* the tree. As they return, we simply update the cached height of every node we visited along the path.

```python
# The update formula

self.height = 1 + max(self.left.height, self.right.height)
```

By doing this, computing the slope (balance factor) of any node becomes an instant $O(1)$ operation, allowing the tree to quickly detect if an insertion just threw it out of balance!

Greedy Algorithms and their application to the **Interval Scheduling** problem.

---

1. What is a Greedy Algorithm?

A Greedy Algorithm solves optimization problems by making a sequence of **locally optimal choices** with the hope that they lead to a **global optimum**.

- **The Rule:** At each stage, the algorithm picks what looks best *right now* based on a specific local criterion. Crucially, it **never goes back to reconsider an earlier decision**.
- **The Benefit:** It drastically reduces the search space, making the algorithm incredibly fast.
- **The Catch:** Many "natural" greedy strategies are completely wrong! Because the algorithm has tunnel vision and cannot backtrack, you must mathematically prove that your local strategy actually guarantees a globally optimal solution.

2. The Problem: Interval Scheduling

Imagine you have a single resource (like a conference room or a supercomputer). You receive a list of booking requests, each with a specific **start time** and **finish time**.

- **The Constraint:** The resource can only be used by one person at a time (no overlapping jobs).
- **The Goal:** Maximize the *total number* of jobs scheduled.

3. The "Wrong" Greedy Strategies

When designing a greedy algorithm for this, our brains naturally jump to a few logical strategies—most of which fail:

1. **Earliest Start Time:** "Pick the job that starts first."
    - *Why it fails:* What if the first job starts at 8:00 AM and lasts for 12 hours? It blocks the room for the entire day, preventing you from scheduling 10 other one-hour meetings.
2. **Shortest Interval:** "Pick the shortest meetings first to fit as many as possible."
    - *Why it fails:* A short 10-minute meeting from 11:55 to 12:05 could overlap with two massive 4-hour meetings (8:00-12:00 and 12:00-4:00). By greedily picking the shortest meeting, you ruin the chance to schedule the other two.
3. **Fewest Conflicts:** "Pick the job that overlaps with the fewest other requests."
    - *Why it fails:* While it sounds smart, there are complex edge-case configurations where picking the job with the fewest conflicts forces a suboptimal chain reaction down the line.

4. The Winning Strategy: Earliest Finish Time

The globally optimal greedy strategy is to always pick the job with the **Earliest Finish Time**.

- **The Logic:** By choosing the job that gets out of the room the fastest, you are freeing up the resource as early as possible for the maximum number of future jobs.
- **The Algorithm:** 1. Sort all jobs by their finish time.
  2. Pick the job that finishes first.
  3. Discard any other jobs that overlap with it.
  4. Pick the next available job that finishes first, and repeat.

5. Proving it Works ("Stays Ahead")

To prove this works, computer scientists use a "Stays Ahead" argument. You compare the Greedy solution to a hypothetical "Optimal" solution step-by-step.

Because the Greedy algorithm strictly picks the earliest finishing jobs, the k-th job in the Greedy schedule will always finish *earlier than or at the exact same time as* the k-th job in the Optimal schedule. It never falls behind, proving it is just as good as the best possible solution.

Greedy Algorithms applied to the **Minimizing Lateness** scheduling problem.

---

1. The Problem: Minimizing Lateness

Imagine you are a freelancer with a list of tasks. For each task i, you know:

- **Duration ($T(i)$):** How long it takes to complete.
- **Deadline ($D(i)$):** The time by which the client wants it finished.

You can only work on one task at a time. If a task finishes *after* its deadline, it is considered "late." The lateness is calculated as the difference between the finish time and the deadline (if it finishes early, the lateness is just 0).

- **The Goal:** You want to find a schedule that minimizes your **maximum lateness** across all tasks. (e.g., You'd rather have 3 tasks be 1 day late than have 1 task be 10 days late).

2. The Failed Greedy Strategies

When trying to solve this greedily, two intuitive strategies immediately fail:

1. **Shortest Processing Time First:** Sorting by $T(i)$ in ascending order.
    - *The flaw:* It completely ignores deadlines! If a short 1-hour task has a deadline of next week, but a 3-hour task has a deadline of today, doing the short task first guarantees the urgent task will be late.
2. **Minimum Slack Time First:** "Slack" is the buffer time you have before you *must* start ($D(i) - T(i)$).
    - *The flaw:* Sorting by smallest slack first can trap you. A long task with a small slack can force you to push several short tasks with slightly larger slacks way past their deadlines.

3. The Optimal Strategy: Earliest Deadline First

The globally optimal strategy is incredibly simple: **Sort the requests by their deadlines $D(i)$ in ascending order**.

- You simply ignore the duration and slack. You process the task that is due the soonest, followed by the next soonest, and so on.
- **Time Complexity:** Because you just need to sort the list and then read off the schedule in order, the complexity is $O(n \log n)$.

4. Proving it Works: The Exchange Argument (Inversions)

How do we mathematically prove this simple strategy guarantees the lowest possible maximum lateness? Computer scientists use an **"Exchange Argument"**.

1. **No Idle Time:** First, we establish that an optimal schedule should have no gaps. If there is a gap, you can just shift all later jobs earlier, which can only decrease lateness.
2. **Defining an "Inversion":** An inversion happens when a schedule puts Job i *before* Job j, even though Job j has an *earlier* deadline ($D(j) < D(i)$).
3. **The Greedy State:** By definition, our "Earliest Deadline" greedy schedule has exactly **zero inversions**.
4. **The Swapping Proof:** Imagine a hypothetical "Optimal" schedule that *does* have inversions. If it has inversions, it must have at least one *adjacent* inversion (two jobs right next to each other out of deadline order).
    - The proof shows that if you **swap** those two adjacent jobs, you remove the inversion, and mathematically, the maximum lateness of the schedule *does not increase*.
    - If you keep repeatedly swapping adjacent inversions, you eventually reach 0 inversions without ever making the schedule worse. Since 0 inversions is exactly our Greedy schedule, it proves our Greedy schedule is just as good as the optimal one!

Huffman Coding, one of the most famous and practical applications of Greedy Algorithms.

---

1. The Problem: Efficient Communication

When computers send messages, they encode characters into binary (0s and 1s).

- **Fixed-Length Encoding:** Standard systems (like ASCII) use a fixed length for every character. For example, if you use 5 bits per letter, a 100-character message always takes 500 bits.
- **Variable-Length Encoding:** In any language, some letters are used far more often than others (e.g., 'E' and 'T' are everywhere, while 'Z' and 'Q' are rare). To save space, we can assign **short codes to frequent letters** and **long codes to rare letters**.

2. The Challenge: Prefix-Free Codes

Variable-length codes introduce a massive problem: **decoding ambiguity**.

Imagine you set `a = 0`, `b = 1`, and `c = 01`. If you receive the message `01`, is it "ab" or is it just "c"? You can't tell.

To fix this, we must use **Prefix-Free Codes**: no character's code can be the prefix (start) of another character's code.

- We can represent a prefix-free code as a **binary tree**.
- Every left branch is a `0` and every right branch is a `1`.
- The actual characters are only placed at the very bottom (the leaf nodes). This guarantees that the moment you hit a character, the code is complete and unambiguous.

3. Huffman's Greedy Strategy (Bottom-Up)

How do we build the optimal tree that minimizes the total message length? The greedy strategy builds the tree from the bottom up.

The Algorithm:

1. Count the frequencies of all characters in your message.
2. Pick the **two characters with the absolute lowest frequencies**.
3. Merge them together to create a new "meta-character" (a parent node) whose frequency is the sum of the two.
4. Put this meta-character back into your pool, and repeat the process until only one root node remains.

By continuously merging the rarest items early, they get pushed deeper and deeper down the tree, meaning they naturally get the longest binary codes. Conversely, highly frequent items are merged last, keeping them near the top with the shortest codes.

4. Time Complexity

To make this algorithm fast, we use a **Min-Heap (Priority Queue)**.

- We insert all n initial characters into a Min-Heap.
- We `delete_min()` twice to get the two lowest frequencies, add them together, and `insert()` the new parent node back into the heap.
- Because heap operations take $O(\log n)$ time, the total complexity to build the optimal Huffman Tree is **$O(n \log n)$**.

> **Historical Fun Fact:** Claude Shannon (the father of information theory) proved mathematical limits on data compression but couldn't find the optimal way to actually build the codes. He and Robert Fano came up with a top-down recursive algorithm (Shannon-Fano) that was decent but flawed. David Huffman was actually a graduate student taking Fano's class when he discovered this flawless, bottom-up algorithm!

Remember, our ultimate goal is to minimize the **maximum lateness** (we want the "worst-case" delay of any single project to be as small as possible).

Here are two tasks:

- **Task A (The Trap):** A massive project with almost no buffer.
  - **Duration:** 100 days
  - **Deadline:** Day 101
  - *Slack:* **1 day** (101 - 100)
- **Task B (The Urgent Quick Job):** A tiny project due almost immediately.
  - **Duration:** 2 days
  - **Deadline:** Day 4
  - *Slack:* **2 days** (4 - 2)

Scenario 1: Minimum Slack First (The Flawed Strategy)

Because Task A has less slack (1 day) than Task B (2 days), the algorithm panics about Task A's lack of buffer and schedules it first.

1. **Do Task A:** Starts at Day 0 $\rightarrow$ Finishes at Day 100.
   - Deadline is Day 101. It is **0 days late**.
2. **Do Task B:** Starts at Day 100 $\rightarrow$ Finishes at Day 102.
   - Deadline is Day 4. It is **98 days late**.

Result: Your maximum lateness is **98 days**. You completely blew past the urgent deadline for Task B because you were tied up with Task A.

Scenario 2: Earliest Deadline First (The Optimal Strategy)

We ignore the slack and strictly look at the deadlines. Task B is due on Day 4, and Task A is due on Day 101, so we schedule Task B first.

1. **Do Task B:** Starts at Day 0 $\rightarrow$ Finishes at Day 2.
    - Deadline is Day 4. It is **0 days late**.
2. **Do Task A:** Starts at Day 2 $\rightarrow$ Finishes at Day 102.
    - Deadline is Day 101. It is **1 day late**.

Result: Your maximum lateness is only **1 day**.

The Takeaway

The "Minimum Slack" strategy fails because it is blind to the absolute calendar dates. It allows a massive, long-term project (Task A) to monopolize your time simply because its internal buffer is tight, forcing you to completely miss the deadlines of smaller, highly urgent tasks (Task B).

DIVIDE & CONQUER :

Divide and Conquer, specifically focusing on the **Counting Inversions** problem.

---

1. The Core Idea: Divide and Conquer

Divide and Conquer is a fundamental algorithmic paradigm. It works by:

1. Breaking a large problem into smaller, disjoint subproblems.
2. Solving those subproblems recursively.
3. Combining the subproblem solutions efficiently to solve the original problem.

Classic examples include **Merge Sort** and **Quicksort**.

2. The Problem: Counting Inversions

You are given a sequence of numbers (e.g., rankings). An **inversion** occurs when two elements are out of order.

- **Mathematical Definition:** A pair (i, j) is an inversion if $i < j$ (it appears earlier in the list) but $A[i] > A[j]$ (its value is strictly greater).
- **Real-World Application (Collaborative Filtering):** If you rank 10 movies, and your friend ranks the same 10 movies, counting the number of inversions between your two lists is a great way to measure how similar your tastes are. 0 inversions means perfect agreement; maximum inversions means exact opposite tastes.

The Naïve Approach:

You could just take every single element and compare it to every element that comes after it. However, this requires nested loops and takes $O(n^2)$ time. We can do better using Divide and Conquer.

3. The Divide and Conquer Strategy

If we split an array into a Left half (L) and a Right half (R), any inversion in the entire array must fall into one of three categories:

1. Both elements are in L (counted recursively).
2. Both elements are in R (counted recursively).
3. **Cross-Inversions:** One element is in L, and the other is in R, where the element in L is strictly greater than the element in R.

4. The Magic Step: Sort and Count

The secret to making this fast is modifying the **Merge Sort** algorithm. By sorting L and R while we count, we can find all cross-inversions in $O(n)$ time during the merge step.

The Merge Logic:

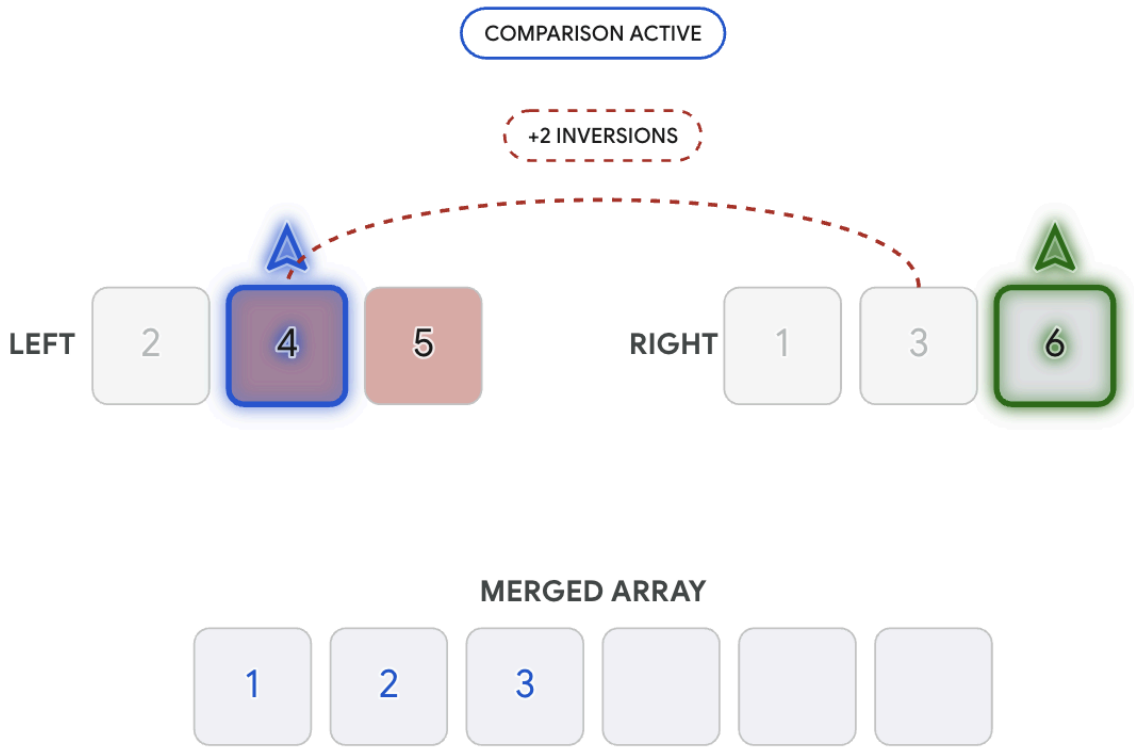
Imagine you are merging sorted array L and sorted array R into a final array B. You have pointers looking at the front of L (i) and the front of R (j).

- If $L[i] \le R[j]$: Everything is in order. We move $L[i]$ into B and do nothing.
- **If $L[i] > R[j]$ (INVERSION DETECTED!):** We move $R[j]$ into B. But here is the trick: Because L is already sorted, if $R[j]$ is smaller than $L[i]$, it is mathematically guaranteed to be smaller than *every single element left in L*.
- Therefore, we don't just add 1 inversion; we add the **number of remaining elements in L** to our inversion count!

5. Time Complexity

Because we are piggybacking on Merge Sort, the mathematical recurrence is identical: $T(n) = 2T(n/2) + n$.

- This solves to **$O(n \log n)$** time.
- *Note:* The actual number of inversions can still be a massive number, up to $O(n^2)$. The brilliance of this algorithm is that we can count an $O(n^2)$ quantity in only $O(n \log n)$ time by counting them in large batches.



Closest Pair of Points problem using **Divide and Conquer**.

---

1. The Problem: Collision Detection

Imagine you are programming a video game or a flight simulator with hundreds of objects (points) moving on a screen. The most basic operation you need to perform is finding the **closest pair of objects** to detect or predict collisions.

- **The Naïve Approach:** You could measure the distance between *every single pair* of points and find the minimum. If you have n points, this requires comparing $\approx n^2 / 2$ pairs, resulting in an **$O(n^2)$ time complexity**.
- **The Goal:** Can we do this faster? Yes, using Divide and Conquer, we can solve this in **$O(n \log n)$ time**.

2. The Divide and Conquer Strategy

Before we do any dividing, we sort all our points twice: once by their X-coordinates (P_x) and once by their Y-coordinates (P_y).

Step 1: Divide (The Split)

We find the middle X-coordinate and draw a vertical line down the screen, splitting our points into a Left half (Q) and a Right half (R).

Step 2: Conquer (Recursive Minimums)

We recursively ask the algorithm to find:

- d_Q: The shortest distance between any two points exclusively in the Left half (Q).
- d_R: The shortest distance between any two points exclusively in the Right half (R).

We let δ (delta) be the absolute minimum of these two distances: $\delta = \min(d_Q, d_R)$.

3. The "Cross-Boundary" Trap

If we just returned δ, our algorithm would be wrong. Why? Because the absolute closest pair in the entire space might be "split" across the boundary—one point sitting just inside the Left half, and the other sitting just inside the Right half.

Step 3: The δ-Band

We only care about cross-boundary pairs if their distance is *strictly less* than our current best distance, δ.

Therefore, we don't have to look at all the points! We only look at points that fall inside a narrow vertical band of width 2δ (going δ to the left of the center line, and δ to the right). Let's call the points in this band S_y.

4. The Geometric Magic (Why it's $O(n \log n)$)

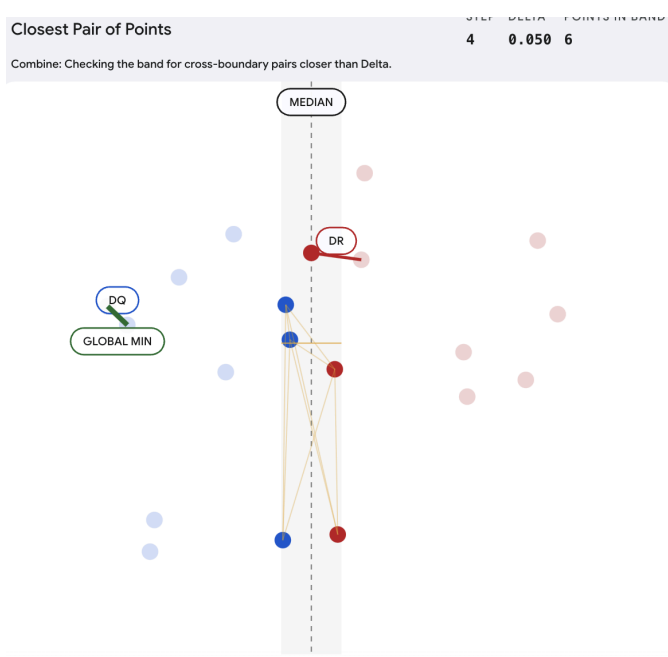
Even if we isolate the points in the band (S_y), comparing all of them to each other could still theoretically take $O(n^2)$ time if all the points clumped up in the band.

This is where the mathematical magic happens:

- By definition, no two points in the *same half* can be closer than δ.
- Because of this strict geometric constraint, the points inside the band are naturally forced to be spaced out.
- If you take any point in the band and look at the next points sorted by their Y-coordinate, you only ever need to check the **next 15 points**. It is geometrically impossible for the 16th point to be closer than δ.

Because we only check a constant number of neighbors (max 15) for each point in the band, scanning the band takes strictly **$O(n)$ time**.

- **Final Recurrence:** $T(n) = 2T(n/2) + O(n)$ (just like Merge Sort), yielding an overall time of **$O(n \log n)$**.



Divide and Conquer approach to **Integer Multiplication**, specifically focusing on the groundbreaking **Karatsuba Algorithm**.

---

1. The Problem: Grade-School Multiplication

How do computers multiply two large numbers? The standard way we learn in elementary school involves multiplying each digit of the bottom number by the entire top number to form "partial products," and then adding them all together.

- **The Bottleneck:** If you are multiplying two n-digit numbers, you have to compute n partial products, each requiring n single-digit multiplications. This means standard multiplication takes **$O(n^2)$ time**. For massive numbers used in cryptography (e.g., 2048 bits), $O(n^2)$ is too slow.

2. The Naïve Divide and Conquer (Still $O(n^2)$)

We can try to split the numbers in half to conquer them recursively. Let's split a number x into its left half x_1 and right half x_0, and do the same for y.

(e.g., if $x = 1234$, then $x_1 = 12$ and $x_0 = 34$).

Mathematically, this looks like:

- $x = x_1 \cdot 10^{n/2} + x_0$
- $y = y_1 \cdot 10^{n/2} + y_0$

If we multiply them:

$x \cdot y = (x_1 y_1) \cdot 10^n + (x_1 y_0 + x_0 y_1) \cdot 10^{n/2} + (x_0 y_0)$

The Flaw: To solve this equation, we have to compute **four** recursive multiplications: $(x_1 y_1)$, $(x_1 y_0)$, $(x_0 y_1)$, and $(x_0 y_0)$.

The recurrence relation becomes $T(n) = 4T(n/2) + O(n)$. If you solve this mathematically, it still results in $O(n^2)$. We haven't saved any time!

3. Karatsuba's Magic Trick ($O(n^{1.58})$)

In 1960, a 23-year-old student named Anatolii Karatsuba found a clever algebraic trick to calculate that exact same formula using only **three** multiplications instead of four!

Here are the 3 multiplications:

1. Compute $A = x_1 \cdot y_1$
2. Compute $B = x_0 \cdot y_0$
3. Compute $C = (x_1 + x_0) \cdot (y_1 + y_0)$

Now, look closely at C. If you expand it:

$C = x_1 y_1 + x_1 y_0 + x_0 y_1 + x_0 y_0$

Notice that the two middle terms $(x_1 y_0 + x_0 y_1)$ are exactly what we need for our original equation! We can isolate them using simple subtraction:

Middle Terms = $C - A - B$

By doing this, we drop the recurrence to $T(n) = 3T(n/2) + O(n)$. This solves to exactly $O(n^{\log_2 3})$, which is roughly **$O(n^{1.58})$**. For massive numbers, $O(n^{1.58})$ is astronomically faster than $O(n^2)$!

4. A Famous Historical Note

In the 1950s, Andrei Kolmogorov—one of the absolute giants of 20th-century mathematics—publicly conjectured that it was mathematically impossible to multiply numbers faster than $O(n^2)$. He brought this up at a Moscow University seminar in 1960.

Karatsuba attended that seminar, went home, and came back just 2 weeks later with this exact algorithm, proving one of the world's greatest mathematicians wrong!

(Note: Donald Knuth later suggested a slight variation using $r = (x_1 - x_0)(y_1 - y_0)$ to avoid numbers carrying over and growing too large during the addition step, which simplifies the computer's memory analysis).

Recursion Trees as a visual and mathematical tool to solve recurrence relations.

---

1. The General Recurrence Formula

When we analyze a divide and conquer algorithm, we usually express its running time $T(n)$ as a recurrence relation.

Almost all standard divide and conquer algorithms follow this general shape:

$$T(n) = aT(n/b) + f(n)$$

- **a**: The number of subproblems you create.
- **n/b**: The size of each subproblem (meaning you divide the input size by b).
- **$f(n)$**: The "overhead" work required to divide the problem and combine the answers at that specific step.

2. Drawing the Recursion Tree

To figure out the total time $T(n)$, we can "unpack" this formula into a tree to see how much work is done at every level of the recursion.

- **The Root:** Represents the initial problem. The work done here is just $f(n)$.
- **Level 1:** The root splits into a children. Each child processes an input of size n/b, so the work for *each* child is $f(n/b)$. The total work across this entire level is $a \cdot f(n/b)$.
- **Level k:** The tree branches out further. At depth k, there are a^k nodes, and each does $f(n/b^k)$ work.
- **The Leaves (Base Case):** The tree stops branching when the input size reaches 1 ($n/b^k = 1$, which means the depth is $\log_b n$). At this very bottom level, there are **$a^{\log_b n} = n^{\log_b a}$** leaf nodes.

3. The Three Scenarios (Intuition for the Master Theorem)

To find the total time, we have to sum up the work done across all levels: $\sum a^k f(n/b^k)$. Depending on the math, this geometric series will behave in one of three ways:

1. **Top-Heavy (Decreasing Series):** The overhead $f(n)$ is so massive that the work done at the root outweighs everything else. The work shrinks as you go down the tree. The total time is just dominated by the root: **$O(f(n))$**.
2. **Uniform (Equal Series):** The splitting perfectly balances the shrinking input size. The total work done at *every single level* is exactly the same. Since there are $\log n$ levels, and each level does $f(n)$ work, the total time is **$O(f(n) \log n)$**.
    - *Example:* Merge Sort $T(n) = 2T(n/2) + n$.
3. **Bottom-Heavy (Increasing Series):** You spawn so many subproblems that the work explodes as you go down the tree. The total time is completely dominated by the massive number of leaf nodes at the very bottom. The total time is **$O(n^{\log_b a})$**.
    - *Example:* Naive Multiplication $T(n) = 4T(n/2) + n \Rightarrow O(n^2)$.
    - *Example:* Karatsuba $T(n) = 3T(n/2) + n \Rightarrow O(n^{\log_2 3})$.

Quick Select and the famous **Median of Medians** algorithm.

---

1. The Problem: Selection

The goal of the selection problem is to find the **k-th largest (or smallest) value** in an unsorted sequence of length n.

- **The Naïve Approach:** You could just sort the entire array in descending order and look at the k-th index. Because the best sorting algorithms take $O(n \log n)$ time, this approach takes $O(n \log n)$ time.
- **The Goal:** Can we do this in linear time, **$O(n)$**? Finding the absolute maximum ($k=1$) or minimum ($k=n$) takes $O(n)$ time because you just scan the array once. But what if we want to find the exact Median ($k = n/2$)?

2. Quick Select (The Recursive Strategy)

To find the k-th element faster than sorting the whole array, we borrow the partitioning logic from **Quick Sort**.

How it works:

1. Pick a **pivot** element and partition the array so that everything smaller than the pivot goes to a `lower` list, and everything larger goes to an `upper` list.
2. Count how many elements ended up in the `lower` list. Let's call this number m.
3. **The Magic Step (Discarding):** Because we know exactly how many items are smaller than the pivot, we know the exact sorted position of our pivot!
    - **Case 1 ($k \le m$):** The answer we are looking for is somewhere inside the `lower` list. We completely discard the `upper` list and recursively search for k in the `lower` list.
    - **Case 2 ($k == m + 1$):** We got incredibly lucky. The pivot *is* exactly the k-th element! We return the pivot and stop.

- **Case 3 ($k > m + 1$):** The answer is in the upper list. We discard the lower list and the pivot. However, because we threw away $m+1$ smaller items, we must update our search target: we recursively search the upper list for the **$k - (m + 1)$**-th element.

3. The Worst-Case Trap ($O(n^2)$)

Quick Select is brilliant because it throws away a massive chunk of the array at every step. However, its speed entirely depends on picking a good pivot.

- **The Bad Pivot:** What if your array is already sorted, and you accidentally pick the maximum element as your pivot every single time? Your lower list will have $n-1$ elements, and your upper list will have 0.
- If this happens repeatedly, you only discard 1 element per step. The recurrence becomes $T(n) = T(n-1) + n$, which results in a terrible **$O(n^2)$ time complexity**.

4. The Rescue: Median of Medians ($O(n)$)

To guarantee that Quick Select runs in $O(n)$ time, we need a mathematical guarantee that our pivot will always split the array relatively evenly (e.g., somewhere in the middle fraction).

In 1973, a group of famous computer scientists invented the **Median of Medians (MoM)** algorithm to find a near-perfect pivot in $O(n)$ time:

1. Divide the unsorted array into small blocks of **5 elements**.
2. Find the median of each tiny 5-element block (using brute force sorting).
3. Take all of those block medians and put them into a new, smaller list M.
4. Recursively call the algorithm to find the *true median* of the list M.
5. Use that "Median of Medians" as your pivot for the main Quick Select partition!

Why blocks of 5? By doing this, the math guarantees that the chosen pivot will always be greater than at least 30% of the array and less than at least 30% of the array. This means the worst-case partition is a 30/70 split. Because $T(n) = T(n/5) + T(7n/10) + n$, the math resolves to a strictly linear **$O(n)$ time complexity**!